

Matching Antipatterns to Improve the Quality of Use Case Models

Mohamed El-Attar James Miller
STEAM Laboratory
Electrical and Computer Eng. Department
University of Alberta
melattar@ece.ualberta.ca
jm@ece.ualberta.ca

Abstract

Use Case modeling is an effective technique used to capture functional requirements. Use Case models are mainly composed of textual descriptions written in natural language and simple diagrams that adhere to a few syntactic rules. This simplicity can be deceptive as many modelers create Use Case models that are incorrect, inconsistent, and ambiguous and contain restrictive design decisions. In this paper, a new methodology is described that utilizes antipatterns to detect potentially defective areas in Use Case models. This paper introduces the tool ARBIUM, which will support the proposed technique and aid analysts to improve the quality of their models. ARBIUM presents a framework that will allow developers to define their own antipatterns using OCL and textual descriptions. The proposed approach and tool are applied to a distributed biodiversity database Use Case model to demonstrate its feasibility. Our results indicate that they can improve the overall clarity and precision of Use Case models.

1. Introduction

UML has been widely accepted as a de-facto standard for producing software blueprints [1]; it supports multiple views and enforces few constraints during modeling. Use Case (UC) modeling is a widespread method to elicit and capture a system's functional requirements. UC models consist of components constructed using natural language and diagrams that contain a small notational set and adhere to few syntactic rules. This formulation is designed to make these models accessible to all stakeholders, including non-technical customers and end-users. Unfortunately, little guidance is provided on constructing semantically correct UC diagrams, leaving the crucial task of making "good" constructive modeling decisions up to the modelers. Hence, the

production of high quality UC artifacts becomes fully dependent on the experience of the modelers; and hence, inexperienced analysts will often produce UC models that are likely to be incorrect, inconsistent and ambiguous, ultimately leading to the production of low quality software systems.

Traditionally, experienced analysts mentor their inexperienced peers. This technique may be viable and effective in a cohesive workplace, with limited staff. However, this situation does not always exist; therefore, one option worth exploring is: can this informal manual process be supplemented by a more structured knowledge management system? This knowledge management mechanism will document previous experiences and knowledge of senior analysts on potentially hazardous techniques and decisions, or bad practices, which will allow inexperienced analysts to gain instant insight into such valuable knowledge. Our approach to the construction of this knowledge management system is to document these "bad practices" in the form of antipatterns.

The remainder of this paper is structured as follows: Section 2 discusses the related approaches and underlines their limitations. In Section 3, we introduce our approach and illustrate it with an example of an antipattern. Section 4 describes the provided tool support (ARBIUM). A case study is presented in Section 5 to further illustrate our approach and ARBIUM. Finally, Section 6 concludes and discusses future work.

2. Related Work on Improving the Quality of UC Models

Researchers have attempted to improve the overall quality of UC models using different strategies; these strategies include:

Table 1. Quality attributes of a UC model

Quality Attribute	Definition	Severity of its absence
Correctness	The UC diagram and textual descriptions must correctly represent the underlying requirements.	Very severe – Designers may build a faulty system.
Consistency	The textual descriptions must conform to the concepts and requirements encoded in the diagram and vice versa.	Very severe – Designers may build a faulty system.
Analytical	The model should not contain any design or implementation decisions, including interface details.	Variable severity – Designers' creativity is restricted. Moreover, it obscures the purpose of the system.
Understandability	The model must be readable and unambiguous. All stakeholders must share a common understanding of the presented functional requirements.	Variable severity – Can be severe if designers cannot understand the model or misunderstand its requirements.

Guidelines: Lilly [4] and Scott [19] produced lists of UC modeling mistakes, many of which can be detected by systematically analyzing the UC diagram. Overgaard et al [11] also presented a list containing eight common UC modeling mistakes. Guidelines for authoring UC descriptions were presented by Cockburn in [17], [18].

Refactoring: Butler et al. introduced the concept of refactoring UC models in [21]; and a list of UC refactorings are introduced in [22]. In [23], an approach for refactoring UC models was described in order to improve their quality regardless of their domain.

Templates: The Requirements UC Tool [5] provides a template for modelers to add information about their UCs. The tool scans these UC descriptions and identifies risky descriptions such as incomplete or weak phrases.

Verification: Ryndina et al. [3] have developed a tool to support their technique Susan, which facilitates the verification of UC models using symbolic model checking. To use this tool effectively, the analyst must input a vast number of values regarding actors, UCs and conditions, which require detailed knowledge about the underlying domain that is usually unavailable until the end of a project. The technique experienced limitations in its support for the UC modeling syntax as defined by OMG [2]. Berenbach [7] has described a set of heuristics and processes for creating large verifiable analysis models. Berenbach made the important distinction between heuristics that are programmatically verifiable and others that require human cognition. The design heuristics presented were hard coded into a tool. However, this restrictive approach is likely to be sub-optimal because:

- Some organizations' policies only allow the use of design heuristics that are developed internally.
- Some organizations only use a subset of UML models to describe and design their software.

Therefore the tool ARBIUM introduced here provides support for analysts to define and verify their own design heuristics and policies. ARBIUM includes a set of rules that can be applied to any UC model to improve its quality by detecting potential defects. ARBIUM also provides a simple framework that will allow designers to code additional rules.

2.1. Quality Aspects of UC Models

Since the quality of a UC model is an essential factor that influences the quality of other artifacts produced throughout the development process, it is important to define quality with respect to UC modeling. Table 1 outlines the most important quality attributes that should be present in a UC model, as stated by [14-16, 19]. Understandability is improved as a result of improving the other quality attributes.

2.2. Why is it Difficult to Produce High Quality UC Models After All?

Several characteristics of UC modeling hamper the production of high quality UC models and severely limit the amount of formal analysis that can be undertaken:

UC Descriptions are Informal: It is difficult to formally analyze the UC textual descriptions since they are defined using natural language. This is an indispensable key feature of UC modeling.

Iterative Development: Commonly UC modeling is utilized within an iterative development process. UC models become more complete with each phase. Hence, a UC model is only complete by the end of a project. Even upon project completion, the UC model still might undergo a few iterations of refinement and finalization.

Hence, many quality improving activities will be undertaken on only partially completed UC models.

UCs Present Only a Subset of a System's Requirements: Only functional requirements are presented in the UC models. Non-functional requirements are outlined in a supplementary specifications document.

Confusion about the UC Modeling Syntax: In 2003, Fuentes et al. [24] identified a large number of errors in the UML 2.0 metamodel; approximately 25 of those errors are specific to UC modeling.

Domain Expertise: Analysts require a certain amount of domain expertise to perform effective analysis of their UC models. For example, certain authoring styles and diagrammatic structures that are suitable for safety critical power plant systems, are most likely not suitable for distributed bank systems. Hence, different analysis techniques are required to effectively analyze UC models representing different domains.

3. Antipatterns of UC Modeling: A New Approach to Improving the Quality of UC Models

This paper focuses on deficiencies that require human cognition to verify that they actually are deficiencies (discussed further in Section 3.2). Structures in the UC diagram that represent discouraged modeling practice can be detected by analyzing or parsing the diagram. The proposed technique can be applied to a UC diagram even if the model is incomplete, allowing the technique to be applied at the early stages of the development cycle.

Table 2. Antipattern template

Name: The title of the antipattern.
Description: A description of the unsound diagrammatic structures.
Rationale: A list of the deceptive reasons as to why the "Fallacious solution" seemed to be the right thing to do.
Consequences: A list of the harmful consequences sustained from applying the "Fallacious solution".
Detection: Where – A guide to the areas where the antipattern can exist. How – Instructions that are used to detect a match for the antipattern.
Improvement: A list of actions that can be performed to convert a "Fallacious solution" into a better solution or avoid the "Fallacious solution".

The approach starts with the capture and documentation of bad modeling habits and decisions present in UC diagrams, such as those discussed in [7-12]. These "issues" are then generalized and developed into a set of antipatterns. These (diagram-based) antipatterns do not assume any knowledge of the underlying domain, which allows their application to all UC models.

3.1. Benefits of Antipatterns

An antipattern is a method that shows the user how to arrive at a good solution from a fallacious solution and how to avoid the specious solution in the future [6]. For a UC description, we believe that the following six categories of information are required to fully describe such behavior (see Table 2).

3.2. How to Use Antipatterns to Search For Risky Areas in UC Models

Antipatterns contain detailed guidelines that will explain to an analyst how to search a UC model for areas that seem risky or debatable. Hence, the approach is defined as "Risk-based" since it involves the identification of high-risk UC structures that are not necessarily deficiencies. If an area in a UC model matches the description of an unsound structure, then that antipattern is matched, and the analyst is then required to judge whether or not corrective measures are required. The process of matching a structure described in an antipattern with the actual UC model is referred to as the *matching* process.

The result of the review process by the analyst may change the structure of the UC diagram. A side effect of changing the diagram is the disappearance of existing antipatterns, which will then no longer be considered, and the surfacing of new antipattern matches. Therefore, the technique must be employed iteratively until all antipatterns are addressed.

3.3. The Two Dimensions of Antipatterns

Discouraged UC structures come in two forms: those that require information about the problem at hand (or the domain of operation); and those that are generic and, on average, are applicable to every UC model. Clearly, researchers can only define antipatterns for the second type of inappropriate structures. But, an ideal process must provide mechanisms for analysts to supply additional antipatterns, which are specific to their problem or situation. This article refers to these two types as domain-dependent and domain-independent

antipatterns respectively and hence the example antipatterns presented in this paper, and found at [13], are domain-independent.

These two types must be further subdivided – with respect to the suitability of the antipatterns to be machine understandable. Clearly, machine understandable antipatterns are preferable as they allow for semi-automation and further machine-driven analysis; this is discussed further in Section 3.5. Antipatterns derived from UC descriptions, rather than UC models, normally present situations where even though parsing it viable, the underlying semantics cannot be machine-captured. Hence, this situation results in a more manual, inspection-like, process driven around natural language components within the antipatterns. Our system ARBIUM uses OCL as the language for describing machine parse-able components in antipatterns. In Section 3.5, the second form (type 2 for Table 3) of the first antipattern provided is an example of a domain-independent antipattern that cannot be described using OCL since it requires the examination of the pre and post-conditions of the UC descriptions. Domain dependent antipatterns that cannot be described using OCL also require the examination of UC descriptions for domain-dependent issues. Table 3 shows the four major situations where antipatterns can be used. For the main situation (1), which this paper focuses on, analysts are provided with a large number of domain-independent antipatterns, available in complete detail at [13], to apply to their UC models. Moreover, the specious structures can be described using OCL and hence tool support can be made available to automate the review process (matching process). For the second situation (2), analysts are provided with a number of domain-independent antipatterns, also available at [13]. However, these antipatterns cannot be described using OCL and therefore the review process needs to be conducted manually. The review process is directed by information supplied in the “Detection” section of the antipatterns. For domain dependent antipatterns that can be described using OCL (3), the antipatterns must be developed by the domain experts. Using the simplified UC metamodel (Fig. 1), analysts can construct OCL statements that accurately describe their antipatterns. As shown later in section 3.5, the majority of antipatterns can be described using one or two OCL statements. For the final situation (4), analysts still need to develop their own antipatterns. However, they will need to utilize a structured review process in order to match such antipatterns. Since UC diagrams relatively do not contain a large number of elements, structuring the review process is far from an overwhelming process.

Table 3. Types of antipatterns

Situation	Tool Support Available	No Tool Support Available
Domain-independent Antipatterns	(1)	(2)
Domain Dependent Antipatterns	(3)	(4)

3.4. Describing Unsound Structures Using OCL

There is an inherent ambiguity when stating any information using natural language. Describing unsound structures stated in antipatterns using natural language is no exception. Using OCL to describe unsound structures eliminates these problems. OCL is mostly used to describe a set of constraints that need to be satisfied in order to confirm that an antipattern did not match. Thus, if the constraints were not satisfied, then an antipattern match is detected. In section 3.5, antipatterns will be augmented with OCL statements wherever possible. OCL is used to describe constraints and invariants for object structures. Therefore, in order to use OCL to describe use case structures, a UC diagram needs to be transformed into an object model. This transformation is made possible since every instance of a UC diagram conforms to the UC metamodel put forward by OMG [2]; i.e. each element in a UC diagram instance instantiates a class or association in the UC metamodel. The UC metamodel describes the syntax rules that all UC diagrams must adhere to in order to determine what structures are syntactically correct. These syntax rules are expressed with OCL statements to remove any ambiguity. Figure 1 shows a simplified version of the UC metamodel that is incorporated in ARBIUM. This version is utilized because the proposed quality improving technique is aimed at novice and inexperienced UC modelers. The original UC metamodel contains over one hundred metaclasses, most of which are not exclusive to UC models. This requires junior modelers to iterate through and understand numerous metaclasses, spread over many metamodels, which makes the task of constructing their OCL statements immensely difficult.

The metamodel provided covers the most commonly used elements in UC modeling. These are: UCs, actors, the include and extend relationships, the association relationship and the generalization relationship between use cases and actors. The metamodel can easily be extended to cover other elements in UC modeling. In instances of our metamodel, each UseCase and Actor

object has a name variable, and an abstract variable to indicate if it is *abstract* or *concrete*. The **ExtendsAt** association class is used to specify an extend relationship linked to a specific extension point. The **extendedUC** role in this association represents the *base* UC, while the **extensionUC** role represents the extension UC and the **extensionLocation** attribute is used to specify the extension point. The extension location is an attribute of the extend relationship between two UCs and hence the **extensionLocation** attribute belongs to the **ExtendsAt** association class, not the **UseCase** class. Therefore, the **ExtendsAt** association needs to have a class of its own. The **ExtendsAt** association differs from the **Extends** association, which only specifies an extend relationship between two UCs with no specified extension points. The **Includes** association is used to specify an include relationship between UCs. The **base** role in this association represents the *base* UC, while the **inclusion** role represents the UC being *included*. The **Specializes_use_case** association is used to specify a generalization relationship between two UCs while the **Specializes_actor** association is used to specify a generalization relationship between two actors. The **Associated_With** association is used to specify an association relationship between a UC (**useCaseEnd** role) and an actor (**actorEnd** role). The multiplicities of all associations are many-to-many. For example, a UC can be *included* many times as well as it can *include* other UCs. The desired cardinalities are only specified when describing antipatterns.

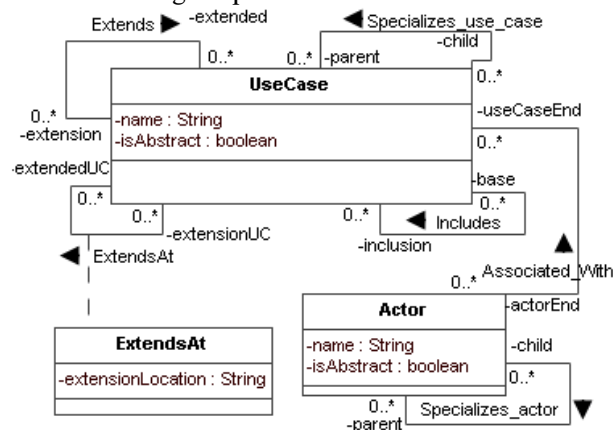


Fig. 1: The simplified version of the UC metamodel used in ARBIUM

As described later in Section 4, the matching process is carried out by the tool USE [20], which utilizes the given metamodel. However, when using an association class in USE, the association class' attributes must all be

initialized. Hence, when using the **ExtendsAt** association class, an extension point name must be given. Therefore, it is not possible to specify an extend relationship with no given extension point using the **ExtendsAt** association and hence the **Extends** association is required. These two associations can be combined by simply adding a Boolean attribute in the **ExtendsAt** association class to indicate whether an extension point exists or not. However, from our point of view, it was believed that separating these two associations would better conform to the semantics of the extend relationship as described in [2], which does not possess a Boolean attribute.

3.5. Examples of UC Modeling Antipatterns

Due to space restrictions only one antipattern is presented in full detail. A large set of antipatterns examples is available at [13].

Antipattern Name: Functional decomposition of UCs

Description: Modelers may divide the functionality of UCs over a number of UCs as means of increasing modularity. Therefore, “smaller” UCs are created to contain portions of the behavior that is supposed to be carried out by a single *base* UC. This concept of modularity may be applied in three forms:

(1) Functional decomposition: Using the *include* relationship - The improper use of the *include* relationship is perhaps the most common form of functional decomposition. Modelers create *included* UCs to contain portions of a complete service offered by a *base* UC. The *included* UCs may represent functions in a program, or menu options. For example, assuming an espresso machine system, a *base* UC called **Prepare Coffee** may *include* three other UCs, such as: (a) Add Sugar, (b) Add Cream or Milk and (c) Pour Hot Water. These *inclusion* UCs are not *included* by any other UC and are not associated with any other element in the UC diagram. Hence, it is evident that the three *inclusion* UCs need to be all executed in order to fulfill the main goal of preparing a cup of coffee.

(2) Functional decomposition: Sequencing UCs using pre and postconditions - When a complete service offered by a *base* UC is subdivided into “smaller” *base* UCs, it is often the case that the “smaller” UCs need to be performed in a particular sequence to properly execute the complete service. However, unlike data flow modeling, UC modeling does not provide any mechanism for UCs to “invoke” or “call” each other.

The reason that such mechanisms are not offered is that a *base* UC, along with any UCs that it *includes*, should be able to perform its intended service without requiring to communicate with other *base* UCs. However, modelers may avoid this limitation by misusing the pre and postconditions stated in UCs to explicitly form a call sequence between them. Therefore, this virtual call sequence would most likely be the result of the UCs being converted into functions. For example, assuming a telecommunications system, a *base* UC that is supposed to make a phone call, is instead divided into “smaller” *base* UCs such as Get Dial Tone, Retrieve Phone Number Dialed, Establish Connection and Ring Destination Phone. The “smaller” *base* UCs are all associated with a Caller actor. The concept of outlining a virtual sequence can be applied in two ways:

- (a) Assuming two UCs named “N” and “N+1” respectively. A call sequence between these two UCs can be virtually enforced by stating in the preconditions of UC “N+1” that UC “N” must successfully terminate. Meanwhile, in a similar fashion, the postconditions of UC “N” states that UC “N+1” must start. Referring back to the telecommunications system, a postcondition of UC Get Dial Tone states that UC Retrieve Phone Number Dialed must start. Similarly, it is stated in a precondition of UC Retrieve Phone Number Dialed that UC Get Dial Tone must successfully end.
- (b) Implicitly, by duplicating the postconditions of UC “N” and restating them as the preconditions of UC “N+1”. This method may be valid at times, however it usually leads to the over-specification of the conditions in UCs.

(3) Functional decomposition: Using the *extend* relationship - The final form of functional decomposition is the improper use of the *extend* relationship. The *extend* relationship is warranted in exceptional situations. However, *extension* UCs are modeled to have knowledge of the *base* UC. Therefore, *extension* UCs are naturally specific to their respective *base* UCs. If an *extension* UC contains general behavior that would be useful to more than one *base* UC, this would be a strong indication that the *extension* UC has degraded into a function. For example, assuming a racquet sports store system, the UC Equipment Damaged *extends* both the Sell Racquet and Sell Ball UCs. The employee must ensure that any equipment being sold is not damaged. The UC Equipment Damaged is initiated whenever any equipment being sold is damaged. However, this structure would indicate that the same procedure would be undertaken regardless of the type of equipment that is damaged. This is most likely not the case.

Rationale (for producing inappropriate structures):

Since functionally decomposed UCs are naturally “smaller”, they will be easier to implement. Since functionally decomposed UCs are naturally “smaller”, they will be easier to understand, implement and test.

Consequences: The purpose of UC modeling is to describe the behavior of a system at the conceptual level. Therefore, the “smaller” UCs created as a result of functional decomposition obscure the real purpose of the system, which significantly reduces the analytical purpose of the UC model and its understandability. Functionally decomposed UCs are often a first cut attempt into embodying design decisions. This may severely limit the creativity of the designers and restrict them to premature design decisions. These “smaller” UCs offer no value to the system’s users if executed individually. Functional decomposition of UCs may lead to more complex descriptions of the interactions between the actors and the UCs, which may make the model prone to inconsistency and incorrectness problems.

Detection: (1) Where – The analyst will need to search for any *inclusion* UCs in the UC diagram. **How** – If an *inclusion* UC is found, the analyst needs to count the number of *base* UCs that *include* it. The antipattern is matched if the *inclusion* UC is *included* only once.

OCL Description:

```
context UseCase
inv NotJustOneInclude:
not (self.base->size = 1)
```

(2) Where – The analyst needs to examine the preconditions and postconditions of each *base* UC depicted in the UC diagram. **How** – (a) If a postcondition for a UC requires the initiation of another UC, then the antipattern is matched. The examination of the post-conditions subcomponents can be performed quickly by searching for the names of other UCs in the model. (b) If a pre-condition of a UC requires the successful completion of another UC, then the antipattern is matched. Similarly, looking for the highlighted names of other UCs can speed up this search process. (c) If a post-condition of a UC and a precondition of another UC, state similar requirements for a particular variable value, then the antipattern is matched. This situation is more difficult to detect because it requires further examination of the contents of the respective UCs.

OCL Description: cannot be described using OCL as it requires examination of the UCs’ textual descriptions

(3) **Where** - The analyst needs to search for any *extension* UCs in the UC diagram. **How** – if an *extension* UC is found, the analyst needs to count the number of UCs it *extends*. If the *extension* UC *extends* more than one UC, the antipattern is matched. At that point, the analyst needs to examine the contents of the *extension* UC to check if it is too generic or specific. If the *extension* UC contains very specific behavior, the

analyst needs to check whether this behavior is in fact suitable for all the *extended* UCs.

OCL Description:

```
context UseCase
inv ExtendingMoreThanOneUseCase:
not (self.extended->size + self.extendedUC->size > 1)
```

Table 4: A summary of antipatterns

Antipattern Name	Brief description
a1. UCs containing common + exceptional functionality	Use cases are used to contain different types of behavior. inv ExtendingMoreThanOneUseCase: not (self.extended->size + self.extendedUC->size > 1)
a2. Accessing a generalized concrete UC	An actor is set to access a parent UC that may contain incomplete functionality inv AccessingGeneralizedUseCaseByActor: not ((not (self.isAbstract)) and self.actorEnd->size > 0 and self.child->size > 0)
a3. Accessing an <i>extension</i> UC	An actor is set to access an <i>extension</i> UC for inappropriate reasons inv AccessingExtensionUseCaseByActor: not((self.extended -> size > 0 or self.extendedUC->size>0) and self.actorEnd->size > 0)
a4. Using <i>extension</i> and/or <i>inclusion</i> UCs to implement an <i>abstract</i> UC	Using the <i>include</i> and <i>extend</i> relationships to implement an <i>abstract</i> UC. <i>Abstract</i> UCs are only properly implemented using the <i>generalization</i> relationship inv UsingIncludeAndExtendToImplementAbstractUC: not((self.isAbstract) and (self.inclusion->size > 0 or self.extension->size > 0 or self.extensionUC->size>0))
a5. Multiple actors associated with a UC	A UC that is associated with more than one actor whom all play the same role when the UC is being preformed inv MultipleActorsAssociatedWithUC: not (self.actorEnd->size > 1)
a6. Multiple generalizations of a UC	A child UC is specializing more than one parent UC to contain common functionality between the parent UCs inv MultipleGeneralizationsOfOneUC: not (self.parent->size > 1)
a7. Duplicating relationships at the <i>generalized</i> and <i>specialized</i> UCs using the <i>include</i> and <i>extend</i> relationships instead.	Inappropriately using the <i>include</i> and <i>extend</i> relationships in addition to the <i>generalization</i> relationship in order to specify common behavior between parent and child UCs inv DupFuncAtChildAndParentUCUsingInclude: not (UseCase.allInstances->forAll (u1 , u2 ((self <> u2) and (u1 <> u2) and (self <> u1))implies (self.inclusion->includes(u2) and u1.inclusion-> includes(u2)))) inv DupFuncAtChildAndParentUCUsingExtend: not (UseCase.allInstances->forAll (u1 , u2 ((self <> u2) and (u1 <> u2) and (self <> u1))implies ((self.extended->includes(u2) or self.extendedUC-> includes(u2)) and(u1.extended-> includes(u2)or self.extendedUC->includes(u2))))
a8. Accessing an <i>abstract</i> UC that is not implemented	An <i>abstract</i> UC needs to be implemented in order for the specified behavior to be performed. inv AccessingUnimplementedAbstractUC: not ((self.actorEnd->size > 0) and self.child->size = 0) and self.isAbstract))

Improvement: UCs that represent portions of a complete service must be reformed into UCs that individually offer a complete meaningful service to a system's user. (1) For the espresso machine system, the functionality contained in the *inclusion* UCs should be merged into the *base* UC Prepare Coffee.

If an *inclusion* UC contains subroutine type behavior, which the modelers would like to distinguish, the behavior of the *inclusion* UC, it should be represented as a part of the *base* UC.

(2) For the telecommunications system, the functionality of the fragmentary *base* UCs should be combined into a single UC (Make A Phone Call), since carrying out a phone call is the real purpose behind the existence of the "smaller" *base* UCs.

(3) For the racquet sports store system, the *extension* UC should be split into two separate *extension* UCs. This allows each of the newly created *extension* UCs to be able to properly handle the exceptional situation occurring at the *base* UC it *extends*. For the case that the *extension* UCs contains general behavior that is specialized by the UCs it *extends*, the *extend* relationship should be replaced with a *generalization* relationship.

Table 4 presents a summary of a number of antipatterns that are located at [13]. The OCL constraints use the context of a `UseCase` object.

4. Automated Risk-Based Inspection of UC Models (ARBIUM)

Even though antipatterns provide information to avoid bad UC modeling habits, they do not control the speed and accuracy of their application. Relying only on human analysts to search through the UC models for risky areas similar to those defined in the antipatterns may still require a lot of time and effort, depending on the skills of the analysts. The cumbersome task of manually matching antipatterns with UC models may significantly hinder the quality of the UC models as many antipatterns may go undetected. ARBIUM is an automated risk-based tool that facilitates the search for antipattern matches. Analysts are then required to judge whether the debatable section requires modification or fixing, or if it is a deliberate modeling decision. Deficiencies that can be found using static analysis mainly occur due to syntax errors in the UC models; such deficiencies can be programmatically detected by software tools, saving analysts time, effort, money and other resources. ARBIUM is not intended to detect static deficiencies. Instead, ARBIUM targets deficiencies that require human cognition.

The debatable low quality structures described in antipatterns are entered into ARBIUM using OCL; by

default ARBIUM can be loaded with the domain-independent antipatterns found at [13]. A given UC model that is created using a UML modeling tool is entered into ARBIUM. The UC model, entered in XML format, is converted into an object model representing the elements of the UC diagram, such as actors, UCs and relationships. At that point, ARBIUM will allow analysts to describe their antipatterns using OCL and to specify any of the predefined antipatterns to be used. The matching process is carried out by the tool USE [20]. USE checks the integrity of information systems against constraints described in OCL. ARBIUM generates two input files for USE: (a) a specification file and (b) a script file. The specification file describes the metamodel structure of a UC diagram and the set of antipatterns specified by the analyst. The script file loads an object representation of a given UC diagram. After completing the matching process, USE presents all antipattern matches for analysts to review. Screenshots and an overview of the operation of ARBIUM are located at [13].

5. The MAPSTEDI System Case Study

The UC model of the MAPSTEDI system was used to evaluate the feasibility of the proposed technique and to demonstrate the application of ARBIUM. The MAPSTEDI system is a distributed biodiversity database system used to merge together the separate specimen collections held by the University of Colorado, the Denver Museum of Nature and Science (DMNS), and the Denver Botanic Gardens (DBG). The UC model was prepared by graduate students at the Ecology and Evolutionary Biology Department (EBIO) at the University of Colorado, and the University of Colorado Museum. The UC model included the UC diagrams as well as all the UC descriptions required to perform a full analysis of the UC model and evaluate the proposed technique. The entire UC model is located at [13]. The UC model is divided into 5 packages (subsystems), each represented with a single UC diagram. Table 5 presents a brief description of each UC diagram. The diagrams contained overlapping entities. That is, UCs and actors that exist in multiple diagrams. Merging UC diagrams that contain overlapping entities may reveal antipattern matches than span multiple diagrams.

As described in section 3.2, the proposed technique was applied iteratively until no further antipattern matches exist. It is important to note that the order of applying the antipatterns is irrelevant. Similarly, the order in which UC diagrams are matched with the antipatterns is also irrelevant. Ultimately, after reviewing all the antipattern matches, the resulting UC model should be the

Table 5. Properties of the UC diagrams of the MAPSTEDI system

UC Diagram	UC count	Actor count	Illustrated Behavior
Database Access	4	2	Database accessibility right
Database Edits	3	3	Editing and updating the database
Database Queries	5	0	Provides a hierarchy of the query services offered by the system
Database Integrator	7	1	Integration of local and remote databases.
Administrative Process	3	2	Shows the administrative responsibilities

same. During the second iteration, an antipattern match was detected that spanned several UC diagrams. Analysis of the UC model and the detection of the antipatterns were performed by the authors of this paper.

Each antipattern match presented in Table 6 was reviewed and checked against its corresponding antipattern definition. After reviewing the antipatterns, six antipattern matches in the first iteration revealed incorrectness in the model; one antipattern match resulted in fixing a consistency issue, all of these issues could have resulted in severe implications. Furthermore, three antipattern matches revealed three separate attempts to display design decisions. The model was changed to provide an analytical representation of the actual requirements.

The second iteration revealed a single incident of incorrectness. In conformance with the fundamental concept of antipatterns, which states that an antipattern

match (diagrammatic or not) does not deterministically indicate the existence of a problem, there was a single antipattern match that was deemed as false positive. A false positive in this context means that the current model was deemed appropriate and no changes were applied. Upon reviewing the antipattern matches from both iterations, the UC model was changed and improved accordingly by the authors.

It is worth noting that there were no antipattern matches that resulted in improving only the understandability aspect of the model. However, as mentioned earlier, the understandability of the model is improved indirectly each time any of the other quality attributes is improved.

6. Conclusions and Future Work

In a UC driven development process, the quality of a UC model greatly affects the quality of other artifacts produced during the development process. Low quality UC models cause defects to propagate through to later phases, where the cost of fixing such defects escalates significantly. We presented the quality attributes of a UC model and devised a technique that can help to improve them, which proved to be effective when performed upon the MAPSTEDI system. The resulting MAPSTEDI UC model is of much higher quality than the original model, with regard to consistency, correctness, understandability and its analytical view of the actual requirements. We are currently developing a large number of domain-independent antipatterns. Antipatterns can be used to improve the quality of other types of UML models; hence we plan to investigate and develop antipatterns for other types of UML models.

Table 6. Antipattern matches in each iteration

	UC Diagram	Antipattern Matched	Quality Improved – Potential Severity
First Iteration Matches	Database Access	Multiple actors associated with a UC	Correctness – very severe
	Database Queries	Functional decomposition: Using the <i>extend</i> relationship	Correctness – very severe
	Database Integrator	Functional decomposition: Using the <i>include</i> relationship	Analytical – non critical
		Functional decomposition: Using the <i>include</i> relationship	Analytical – non critical
	Database Edits	Functional decomposition: Using the <i>include</i> relationship	Analytical – non critical
		Accessing an <i>extending</i> UC	Correctness and Consistency – very severe
	Administrative Process	Multiple actors associated with a UC	Correctness – very severe
		Multiple actors associated with a UC	Correctness – very severe
Multiple actors associated with a UC		Correctness – very severe	
Second Iteration Matches	Merged UC Diagram	Accessing a <i>generalized concrete</i> UC	Correctness – very severe
		UCs containing common and exceptional functionality	False positive

The proposed technique has greatly benefited from tool support. ARBIUM facilitates the matching process and provides a framework for analysts to specify their customized antipatterns. ARBIUM can be further developed to support other types of UML models and become integrated into a large-scale UML development tool.

Some current limitations should be addressed for possible improvements in future work. Firstly, at least some basic knowledge of meta-modeling and OCL is required by modelers in order to utilize ARBIUM. As illustrated in this case study, UC models are not necessarily built by UC modeling experts, and having such experts on hand will not always be the case. Therefore, the antipatterns approach provides simple step-by-step guidance to novice modelers to allow them to apply the presented technique. This limitation however is not as severe as it is expected that UC models built by novice modelers are simple in structure, containing a relatively low number of UCs, which will allow them to search for and apply the antipatterns manually. On the other hand, complex UC models with up to 1000 UCs are expected to be built by large organizations that certainly should have expert analysts on hand who are at least familiar with meta-modeling and OCL, and should be capable of utilizing ARBIUM to its full extent. Such analysts would be able to extend the simple UC meta-model shown in Fig. 1 and create antipatterns using OCL to cater to their organization's specific needs. Secondly, the meta-model only tackles diagrammatic antipatterns and does not directly cover the UC descriptions; manual evaluation of the descriptions is required.

Acknowledgements

We would like to acknowledge Dr. Lionel Briand and Dr. Yvan Labiche for their invaluable comments and feedback towards this work. We would also like to acknowledge MagicDraw for their support.

References

- Booch, G., Rumbaugh, J., Jacobson, I.: *The Unified Modeling Language User Guide*. Second Edition. Addison-Wesley, (2005)
- OMG 2003, "UML Superstructure Specification", Object Management Group, <http://www.omg.org/docs/ptc/03-08-02.pdf>, 2003.
- Ryndina, O., Kritzing, P.: Improving Requirements Specification: Verification of UC Models with Susan. Technical Report CS04-06-00, Department of Computer Science, University of Cape Town (2004).
- Lilly, S.: UC Pitfalls: Top 10 Problems from Real Projects Using UCs. Proc. of TOOLS USA '99, IEEE Computer Society, (1999).
- McCoy, J.: Requirements UC tool (RUT). Companion of the 18th annual ACM SIGPLAN. OOPSLA'03. 104-105 (2003)
- Brown, W., Malveau, R., McCormick, h., Mowbray, T.: *Refactoring Software, Architectures, and Projects in Crisis*. Wiley. 1998.
- Berenbach, B.: The Evaluation of Large, Complex UML Analysis and Design Models. Proceedings of the 26th ICSE'04 0270-5257/04. (2004)
- Armour, F., Miller, G.: *Advanced UC Modeling*. Addison-Wesley (2000)
- Kulak, D., Guiney, E.: *UCs: Requirements in Context*. Addison-Wesley (2000)
- Bittner, K., Spence, I.: *UC Modeling*. Addison-Wesley, MA (2002)
- Overgaard, G., Palmkvist, K.: *UCs: Patterns and Blueprints*. Addison-Wesley (2004)
- Guidelines to use case modeling. Last accessed 2006. http://www.zoo.co.uk/~z0001039/PracGuides/pg_use_cases.htm
- STEAM Laboratory website. Last updated May, 2006. http://www.steam.ualberta.ca/main/research_areas/AntiPatterns.htm
- Jaaksi, A. Our Cases with UCs. *Journal of Object-Oriented Programming*, Vol. 10, No. 9, pp. 58-64. (1998)
- Firesmith, D.G. UC Modeling Guidelines. Proc. of TOOLS 30. IEEE Computer Soc. (1999)
- Ben Achour, C., Rolland, C., Maiden, N.A.M. & Souveyet, C. Guiding UC Authoring: Results of an Empirical Study. Proc. IEEE Symposium on Requirements Engineering, IEEE Comput. Soc, Los Alamitos, CA, USA. 1999.
- Cockburn, A. Structuring UCs with Goals. Technical report. Human and Technology, 7691 Dell Rd, Salt Lake City, UT 84121, Ha.T.TR.95.1. 1995.
- Cockburn, A. *Writing Effective UCs*. Addison-Wesley, 2000.
- Rosenberg, D. & Scott, K. *UC Driven Object Modelling with UML*. Addison-Wesley, 1999.
- Gogolla M., Bohling J., and Richters M.: Validation of UML and OCL Models by Automatic Snapshot Generation. Proc. 6th Int. Conf. Unified Modeling Language (UML'2003). Springer, Berlin. 2003
- Butler, G., Xu, L.: Cascaded refactoring for framework evolution, Proceedings of 2001 Symposium on Software Reusability, ACM Press, 2001, pp. 51-57.
- Rui, K., Butler, G.: Refactoring UC models: The metamodel. Proc. Of (ACSC2003), Vol. 16. (2003)
- Ren, S., Rui, K., Butler, G.: Refactoring the Scenario Specification: a Message Sequence Chart Approach. Proc. 9th OOIS 2003. p 294-298
- Fuentes, J.M., Quintana, V., Llorens, J., Genova, G., Prieto-Diaz, R. Errors in the UML Metamodel? *ACM SIGSOFT Software Engineering Notes*. Volume 28 Issue 6. 2003.